17. Context API in React

▼ What is React Context API?

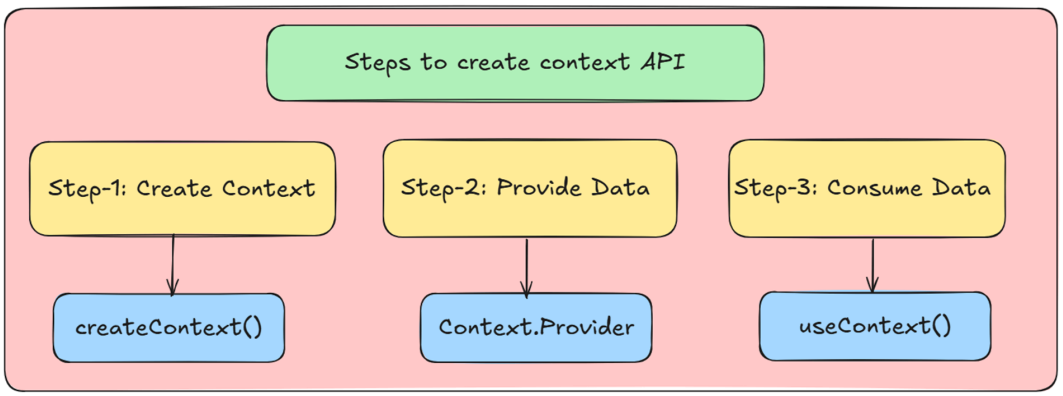
- Context API is a new feature added in version **16.3** of React.
- **The context API provides a way to pass data through the component tree without having to pass props manually at every level.**
- Context = **Global storage**
- Context API is nothing but a global container where we store data, and whichever component needs it can directly access it.
- Context API is used to avoid `prop drilling` in React.

▼ What is Prop Drilling?

- Prop Drilling means passing data from a parent component to a deeply nested child component through multiple intermediate components that do not need that data.
- **Prop Drilling = Unnecessary passing of props through many components**
- **Solution to Prop Drilling → Context API**✅

▼ How to create the Context API / Provider

- In the context API, we have two important things:
  1. Provider
  2. Consumer
- **Steps to create the context API:**



▼ Creating Context:

- First, you need to create a context using the `createContext` function from the `react` module.
- This function returns an object with two components: `Provider` and `Consumer`.
- **Code:**

```
import { createContext } from "react";

let MyContext = createContext();
console.log(MyContext); //returns an object
```

- **Output**:

```
{$$typeof: Symbol(react.context), _currentValue: undefined, _currentValue2: undefined, _threadCount: 0, Provider: {…}, …}
  $$typeof: Symbol(react.context)
  Consumer: {$$typeof: Symbol(react.consumer), _context: {…}}
  Provider: {$$typeof: Symbol(react.context), _currentValue: undefined, _currentValue2: undefined, _threadCount: 0, Provider: {…}, …}
  _currentRenderer: null
  _currentRenderer2: null
  _currentValue: undefined
  _currentValue2: undefined
  _threadCount: 0
  [[Prototype]]: Object
```

▼ Creating Provider Component:

- The `Provider` component is responsible for making the data (state) available to its descendants.
- It takes a `value` prop, which is the data you want to share.
- **Code:**

```
import React from "react";
import { createContext } from "react";

let MyContext = createContext();
console.log(MyContext); //returns an object

const MyContextProvider = ({ children }) => {
  const myData = "Hello from Context!";

  return <MyContext.Provider value={myData}>{children}</MyConte
xt.Provider>;
};

export default MyContextProvider;
```

- In this example, the `value` prop is set to `myData`, which can be any data type – it could be a state, an object, or any other piece of information you want to share.

▼ Wrapping Components:

- Wrap the components that need access to the shared data with the {component.
- **Code:**

```
import React from "react";
import MyContextProvider from "./Context API/MyContextProvider";

const App = () => {
  return (
    <div>
      <MyContextProvider>
        {/* Your components go here */}
      </MyContextProvider>
    </div>
  );
};
```

```
export default App;
```

▼ Consuming Context:

- To access the shared data within a component, use the `useContext` hook or the `Consumer` component.
- **Code:**

```
import React, { useContext } from "react";
import { MyContext } from "./MyContextProvider";

const MyComponent = () => {
  // Consuming the context
  let myData = useContext(MyContext);
  console.log(myData);
  return (
    <div>
      <h2>Data Coming from the MyContextProvider</h2>
      <p>{myData}</p>
    </div>
  );
};

export default MyComponent;
```

- **Output:**

Data Coming from the MyContextProvider

Hello from Context!

▼ What is `useContext` Hook?

- The `useContext` hook is one of the hooks introduced i n React 16.8.

- The `useContext` is a React hook that allows a component to directly access data from the Context API without using props or the `< Consumer >` component.
- In Simple words, the `useContext` **hook is used to access the global data stored in context.**
- **Syntax**: **const value = useContext(MyContext);**

▼ Common Use Cases for the Context API

- The Context API is versatile and can be used in various scenarios where managing state and sharing data across multiple components is necessary. Here are some common use cases:
  1. **Global State Management in medium to large apps:** The Context API can handle global state management like cart items in an e-commerce app or the currently playing song in a music app.
  2. **Authentication Management:** Using the Context API and other solutions like it to manage the auth state is a common use case for it. States like the current user and auth tokens can be shared across the application using the Context API. This allows any component to access the user authentication status and perform actions like login and logout, or display certain items based on the state.
  3. **Theme Management:** Another popular use case for the Context API is theme management (dark mode and light mode toggles). You can do this by storing the theme state in a context, and then accessing and updating the theme in any component without having to pass props through multiple layers.
- Other use cases are localization, user preferences like notification settings, open, close, and toggle states of a modal, API request management, breadcrumbs navigation, step progress, and any other point where the state is involved.